

Lecture 3 Linear Regression and Data Generating Distribution

Announcements

- Homework 1 due next Wednesday 9/16
- Canvas Quiz
 - No automated feedback for each attempt
 - If you are not sure—> OH
 - unlimited attempts + most recent score

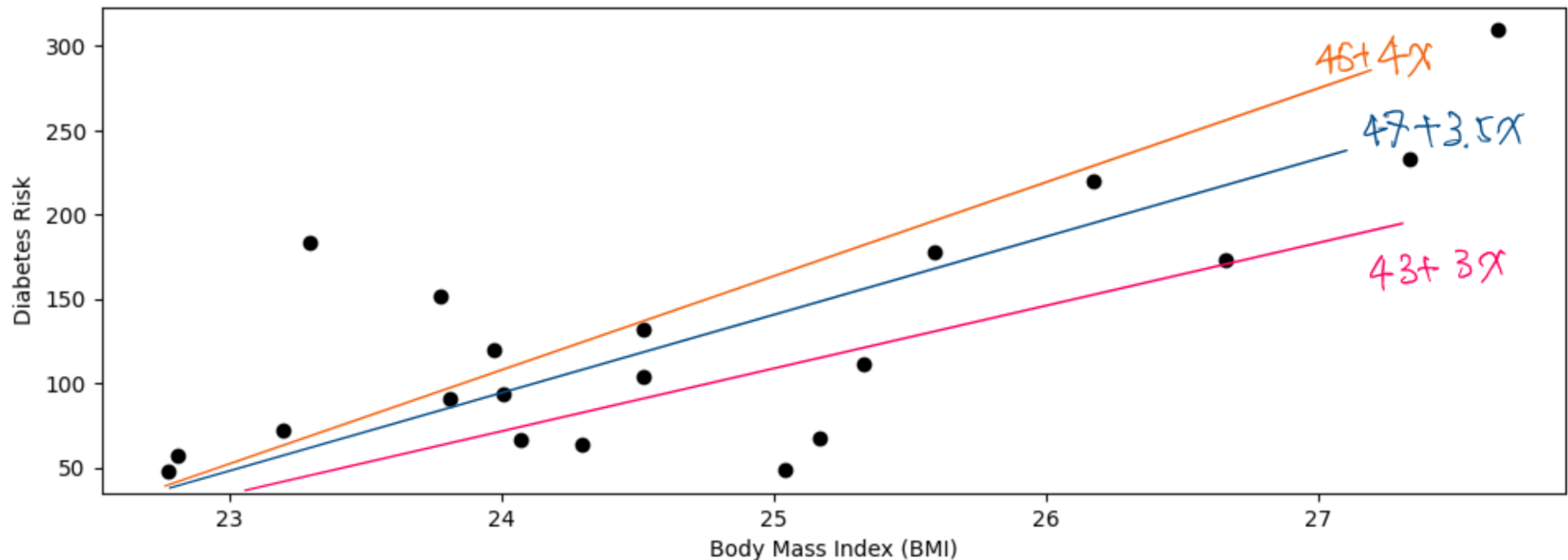
Machine Learning Basics

- Input data
- Output (targets task, target values)
- Model class
- Loss function/objective function
- Learning rule/optimizer

How Linear Regression Works



- Decide to use the linear regression model class to predicting diabetes risk using BMI
- Each Line is a model and forms a valid hypothesis of the true model



$\mathcal{H} = \{f_{\theta}(x) \mid \theta_0, \theta_1 \in \mathbb{R}\}$. $\rightarrow$ model class associated with this linear regression.

Objective Functions

- Find the model with the **best fit** in my model class
- Best fit: minimizing the distance between the predicted label and the true label

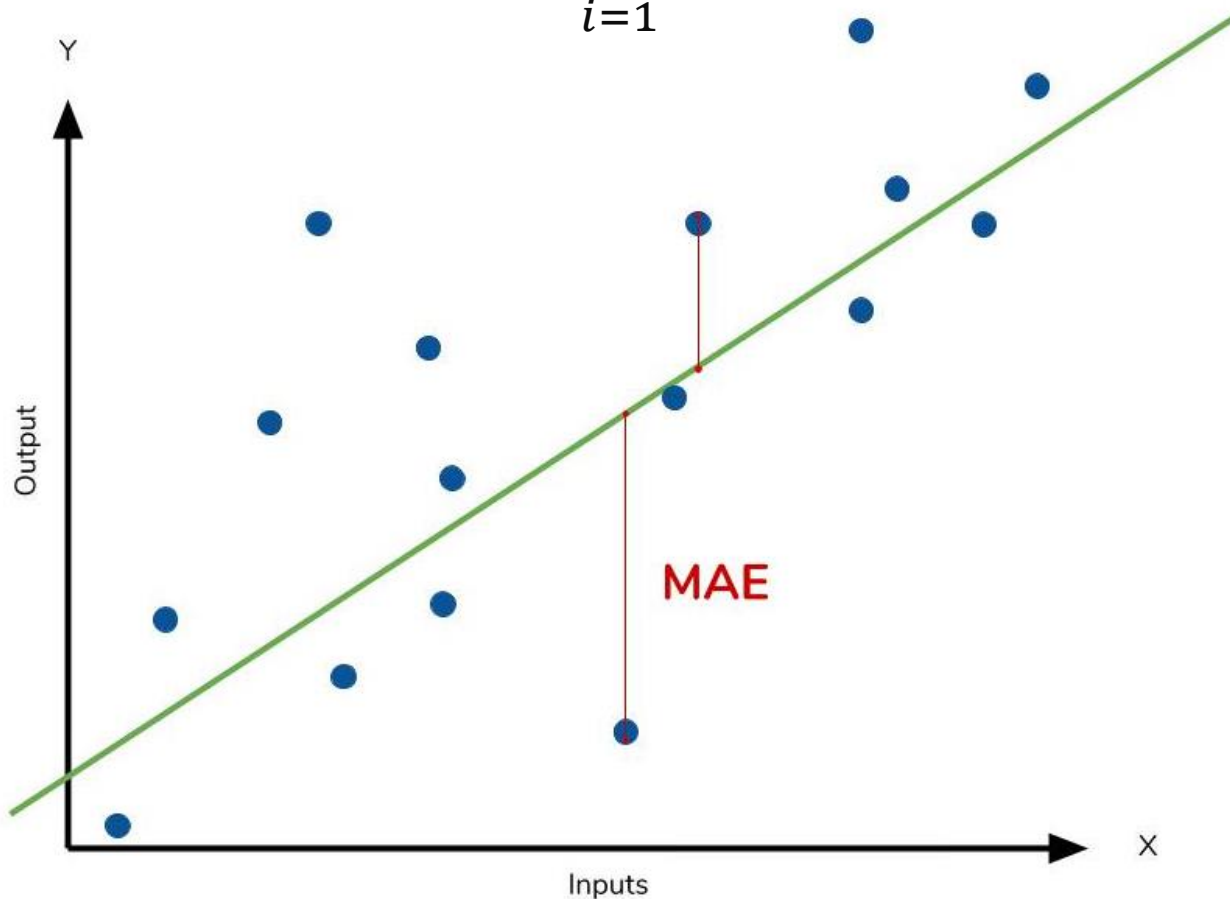
$$loss_{\theta}(\hat{y}^{(i)}, y^{(i)})$$

- Absolute Error
- Mean Squared Error
- Root Mean Squared Error

Regression objective functions

- Absolute error

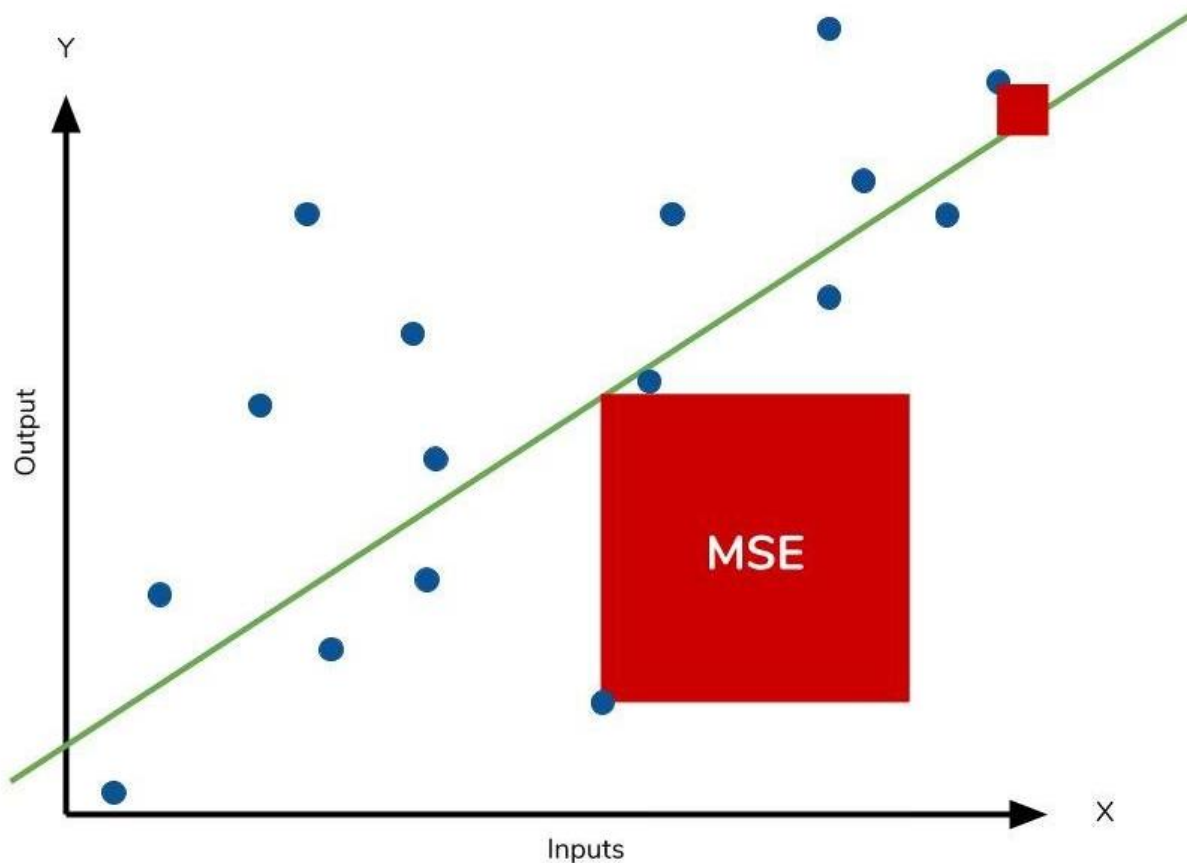
$$\text{loss}_{\theta} = \frac{1}{n} \sum_{i=1}^n |f_{\theta}(x^{(i)}) - y^{(i)}|$$



Regression objective functions

- Mean squared error (most common in linear regression)

$$\text{loss}_{\theta} = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



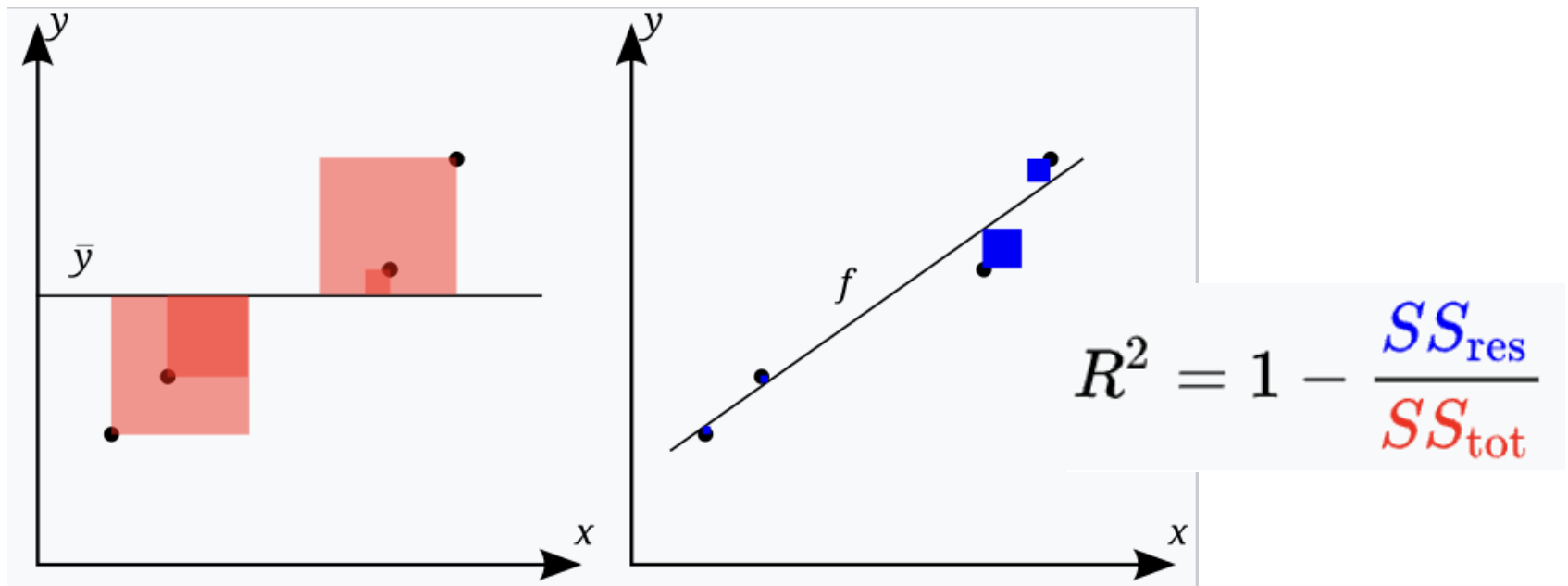
Regression objective functions

$$\text{loss}_\theta = \sqrt{\frac{1}{n} \sum_{i=1}^n (f_\theta(x^{(i)}) - y^{(i)})^2}$$

- RMSE puts the error back in the **units of the label**
- What is a good RMSE performance?
 - Depending on the variance of the data
- Coefficient of determination (R^2): evaluation metric
 - Not a loss function

Coefficient of determination (R^2)

- Measures the amount of variance explained by the linear regression model



- R^2 moves closer to 1 as the fit becomes better
- Normalized metric that allow goodness of fit comparison across different datasets

Training / Optimizer

- Once we fix a model class (parameterize the model)
 - Infinite number of values that parameters can take
 - How these parameters should change?
 - **A: Find the minimizer of the objective/loss function**

Notation

- An optimizer takes a loss function and a model class $\mathcal{M}$
- Finds a model with the smallest value of the loss function (or best “fit” to the data)

$$\min_{f \in \mathcal{M}} \text{loss}(f)$$

- When the model is parameterized by θ , we minimize over the parameter space Θ

Notation: Design Matrix

Represent the entire dataset as one matrix $X \in \mathbb{R}^{n \times d}$, of the form:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & & & \\ x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} \end{bmatrix} = \begin{bmatrix} - & (x^{(1)})^\top & - \\ - & (x^{(2)})^\top & - \\ & \vdots & \\ - & (x^{(n)})^\top & - \end{bmatrix}.$$

Linear Regression Admits Closed-Form Solution

Optimization Problem

- Minimize the sum of the squared errors

$$\min \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}} \left(\left(\theta_0 + \theta_1 \cdot x_1^{(i)} + \theta_2 \cdot x_2^{(i)} + \cdots + \theta_d \cdot x_d^{(i)} \right) - y_i \right)^2$$

Succinctly (with linear algebra): $\min_{\theta} \|\theta^\top X - y\|_2^2$

- With $x_0 = 1$.

Linear Regression Admits Closed-Form Solution

- Linear regression admits closed-form solution under mean squared loss

$$\begin{aligned}\ell(\mathbf{y}, \hat{\mathbf{y}}) &= \|\theta^T X - \mathbf{y}\|_2^2 \\ &= \sum_{i=1}^n (y_i - \theta^T \mathbf{x}^{(i)})^2 \\ &= (\mathbf{y} - \theta^T X)^T (\mathbf{y} - \theta^T X)\end{aligned}$$

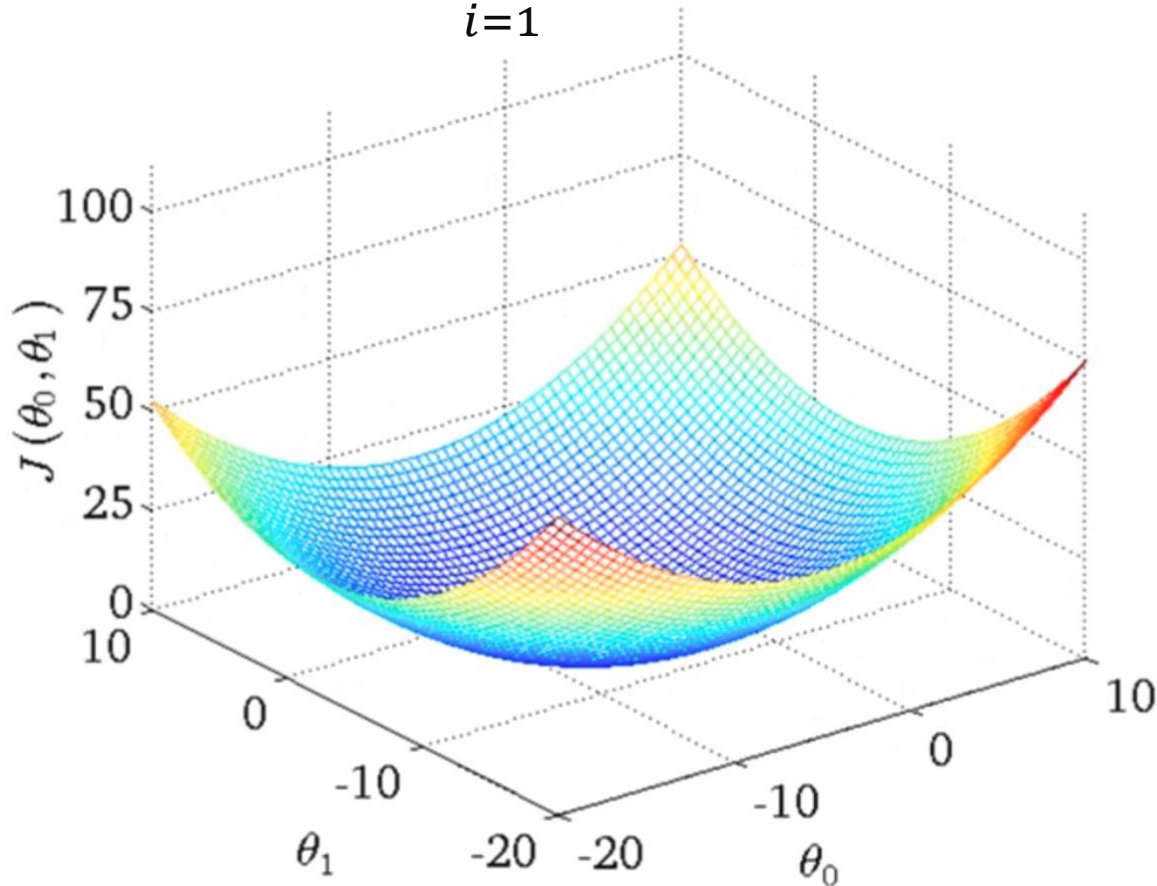
- The derivation was presented in the first review session

```
model.fit(X_train, y_train)
```

Regression objective functions

- Mean squared error (most common in linear regression)

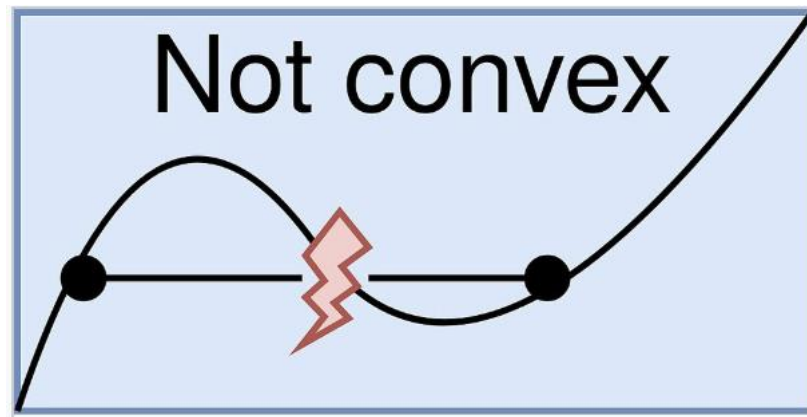
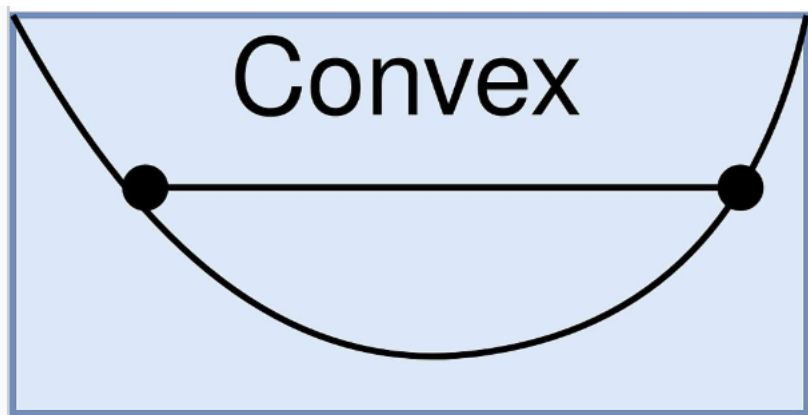
$$\text{loss}_{\theta} = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



Convex functions

For all $0 \leq t \leq 1$ and all $x_1, x_2 \in X$:

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$



Training / Optimizer

In general, many models are trained by stochastic gradient descent (SGD)

- Gradients (multivariable variable calculus)
- Programs that can perform automatic differentiation
- Pytorch

Training / Optimizer

Gradient descent

- Gradient: generalization of derivatives to vectors and matrices
- Can be as an optimization technique to optimize **any** differentiable loss functions
- Given the loss function, it takes the partial derivatives
 - Informs us of the directions where the loss increases/decreases most rapidly
- Move the parameter in the direction where the loss goes down (direction of negative gradient)

Training / Optimizer

- Random initialization
- Repeat this descent process iteratively:
 - Grab data point (x, y) from dataset
 - Pass x into ML model, generating $f(x)$
 - Calculate $\text{loss}(f(x), y)$
 - Update weights w in the direction that lowers the loss

What is the most common objective/loss function for linear regression?



Least squares

0

Gradient Descent

0

Mean squared error

0

R^2

0

Select all that can be used as an evaluation metric for linear regression:

 0

Least Squares

0

Root Mean Squared Error

0

R^2

0

Mean Squared Error

0

Accuracy

0

Training / Optimizer: Guarantees

- Optimality:
 - Guaranteed to reach global optimum if
 - the loss function is convex and
 - with suitable step sizes
- If the function is not convex, i.e., multiple optimum exists
 - the stochastic part will nudge the loss function out of the local minimum point
 - Hopefully, you reach a global optimum, but there is no guarantee

The loss of my linear regression is high on my dataset, what could have gone wrong?

✓ 0

I picked the wrong model class, i.e., the underlying data generating mechanism is highly nonlinear

0

the loss function got stuck at a local minima

0

I did not preprocess my data well

0

The model class is too small since linear regression only contains a finite number of hypotheses

0

The features in my dataset are not linearly independent of each other

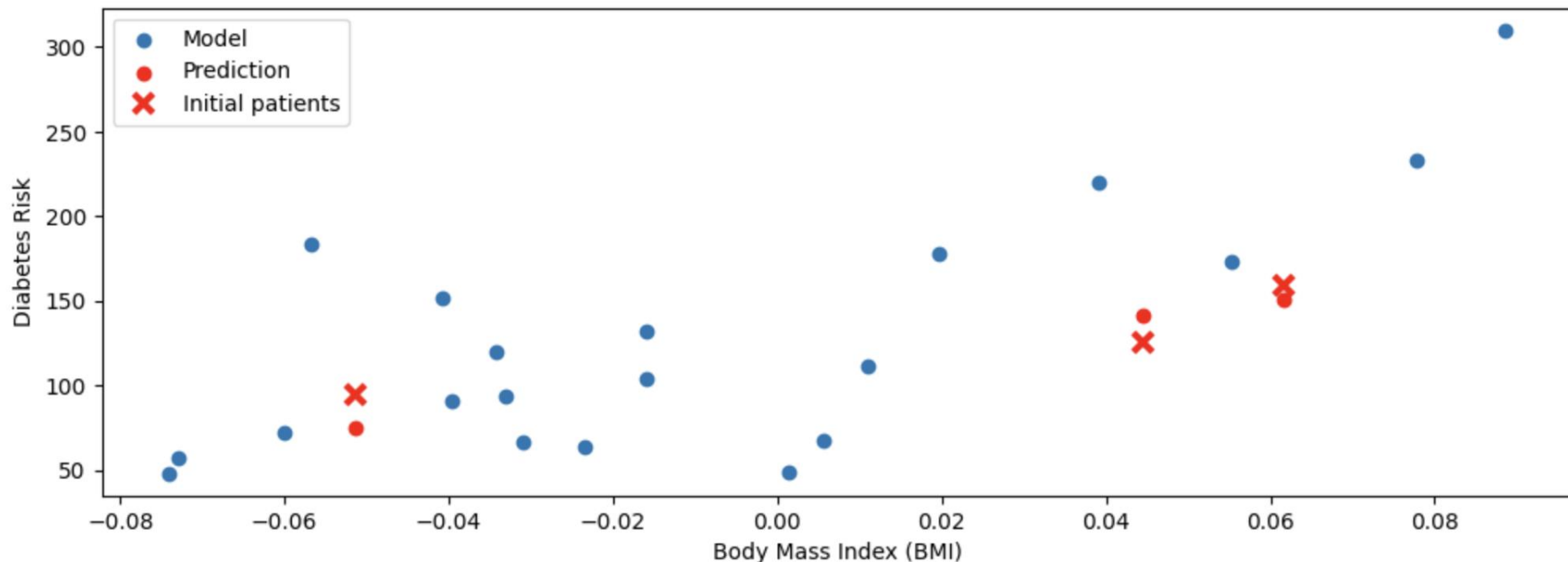
0

All of the above

0

Generalization

- After training, model accurately classifies images we have seen
- But we want to perform well on previously unseen images!



Objective of Learning

- Find the model within the **model class** that **best fits** the data
- Achieved through
 - Defining the loss/objective function
 - Use of optimizer to find the **set of parameters** that **minimizes** the loss function

Components of a Supervised Machine Learning Problem

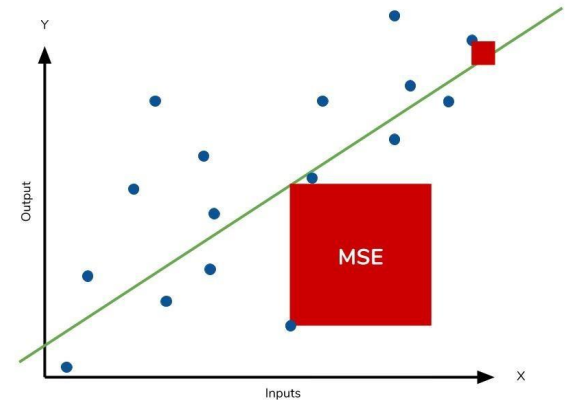


Output: a predictive model that maps inputs to targets

- It can predict targets on new inputs

Recall Regression: Mean squared error (most common in linear regression)

$$\text{loss}_{\theta} = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x^{(i)}) - y^{(i)})^2$$



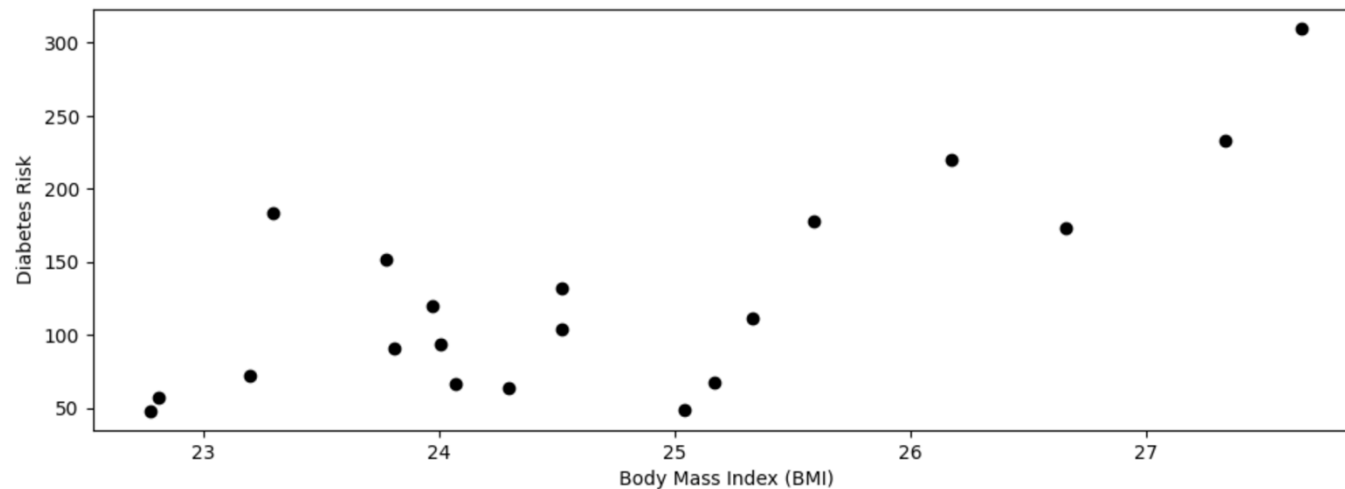
Next

- Automated (universal) way of optimizing an objective/loss function
- Before that, the philosophy of thinking about data and what machines are learning:
 - Data generating distribution and empirical risk

Model Class: Modeling the Data Generating Distribution

Randomness in the attributes of the data

- X : BMI, Y : diabetes risk
- X and Y are now **random variables**
- Random variable (RV): mathematical ways of
 - Model randomness
 - Thinking and quantifying **random events**



Probability Refresher: Random Variables

- The BMI of the next data point that I will see is **random**: I am not certain what value it will take
- To formalize the above idea in math, we need to use the notation of a random variable
 - to define the set of all possible BMI that a person can take (sample space)
 - to define the likelihood that a person has that BMI (probability)
 - to talk about the distribution of the random variable/data (density function)

Random Variables: Example

- A RV, $X: \Omega \rightarrow E$, is a **mapping** from ***the set of possible outcomes in a sample space, Ω*** , to a *measurable space, E* .
- X : **outcome** we observe from a single coin toss

○ $X(\text{image of heads}) = H$

○ $X(\text{image of tails}) = T$

Random Variables: Example

- A RV, $X: \Omega \rightarrow E$, is a **mapping** from *the set of possible outcomes in a sample space*, Ω , to a *measurable space*, E .
- X : the **number of heads** that we observe from two coin tosses

○ $X(\text{) = 2$

○ $X(\text{) = 1$

Random Variables: Example

- A RV, $X: \Omega \rightarrow E$, is a **mapping** from ***the set of possible outcomes in a sample space, Ω*** , to a *measurable space, E* .
- X : **whether head had appeared** from two coin tosses

○ $X(\text{) = 1$

○ $X(\text{) = 1$

Probability Refresher: Probabilities Distributions

- Random variables are something that we construct to model random events
- The **distribution of random variables** allows us to define a *probability distribution* that quantifies how likely an event is going to occur in the outcome sample space.
- Probability distribution satisfies the following axioms:
 - $P(\Omega) = 1, P(\emptyset) = 0$
 - $0 \leq P(\omega) \leq 1$ for all $\omega \subseteq \Omega$
 - Satisfy countable additivity property

Probabilities Distributions: Example

Probability distribution satisfies the following axioms:

- $P(\Omega) = 1, P(\emptyset) = 0$
- $0 \leq P(\omega) \leq 1$ for all $\omega \subseteq \Omega$
- Satisfy countable additivity property

Flipping a fair coin once

- $P(X = T) = 0.5, P(X = H) = 0.5$
- $P(\text{observes nothing}) = 0$
- $P(\text{observe either } X = T \text{ or } X = H) = P(X = T) + P(X = H) = 1$

Probability function is something you can construct!!

Probability Refresher: Random Variables

- Attributes of data can be thought as RVs
- Data are drawn from **data generating distributions**
- The dataset contains the realization of RVs

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019908	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068330	-0.092204
2	0.085299	0.050680	0.044451	-0.005671	-0.045599	-0.034194	-0.032356	-0.002592	0.002864	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022692	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031991	-0.046641

Probability Refresher: Random Variables

- Attributes of data can be thought as RVs
- Data are drawn from **data generating distributions**
- The dataset contains the realization of RVs
- For example:
 - The BMI of the entire population forms a distribution
 - I can think of each person's BMI as randomly sampled from this distribution independently
 - i.e., the BMI satisfies the **independent and identically distributed (iid)** assumption

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019908	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068330	-0.092204
2	0.085299	0.050680	0.044451	-0.005671	-0.045599	-0.034194	-0.032356	-0.002592	0.002864	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022692	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031991	-0.046641

Common IID Assumption

- Independent
 - The values that each sample takes are independent of each other
 - My BMI has no effect on your BMI
 - Really?
- Identically distributed
 - The samples are taken from the sample distribution
 - My BMI and your BMI are sampled from the same distribution
- IID is **not** always assumed by ML methods

Model Class: Modeling the Data Generating Distribution

- Dataset sampled independent and identically distributed (iid) from distribution P .
 - $(X^{(i)}, y^{(i)}) \sim P$
- Example:
 - X : BMI, Y : diabetes risk

Data Generating Distribution

- Dataset sampled independent and identically distributed (iid) from distribution P .
 - $(X^{(i)}, y^{(i)}) \sim P$
- Example:
 - X : BMI, Y : diabetes risk
 - P is described by
$$Y = \alpha + X + \varepsilon, X \sim N(168, 30), \varepsilon \sim N(0, 20)$$
- The iid assumption
 - Algorithmic guarantees

Why Modeling the Data Generating Process?

- Enable us to describe how ML algorithms are learning
 - In the language of
 - Generalization error, **True risk**, **Empirical risk**...
- Model randomness/uncertainty in the dataset
- Can think of data as generated according to some unknown function f :

$$Y = f(X) + \epsilon$$

- Use probability and statistics to analyze ML algorithms

Select all statements that are true

0

To run linear regression on a dataset, we would require the underlying data-generating mechanism (of the dataset) to be linear

0

Oftentimes, the data-generating mechanism of a dataset is unknown, but it is easy to make guesses on which family of the distribution the dataset is generated from

0

In general, we expect linear regression to work well if the underlying data-generating mechanism (of the observed variables) is linear

0

Deep learning methods often achieve superior performance in practice, as their model classes are more expressive than those of prior approaches.

0

None of the above

0

Question: Linear Data Generating Process V.S. Linear Regression

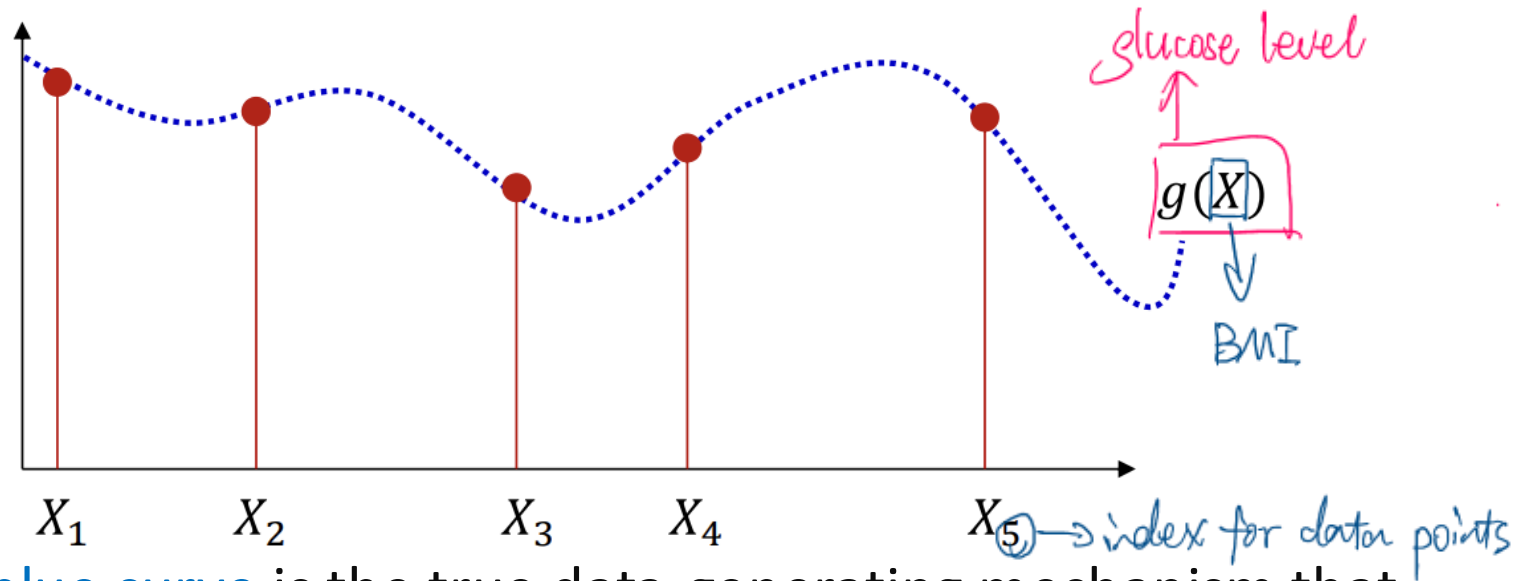
- When the data generating process is indeed linear, what does R^2 converge to in a linear regression when the number of samples grows to infinity?
- Data generating process:
 - $Y = \alpha + \beta X + \varepsilon, X \sim N(168, 30), \varepsilon \sim N(0, 20)$
- Run linear regression: $Y^{(i)} = \alpha + \beta X^{(i)}$
- What does R^2 converge to? (Homework 2)

$$R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{RSS}{TSS}$$

RSS: sum of the squares of residuals; TSS: total sum of squares

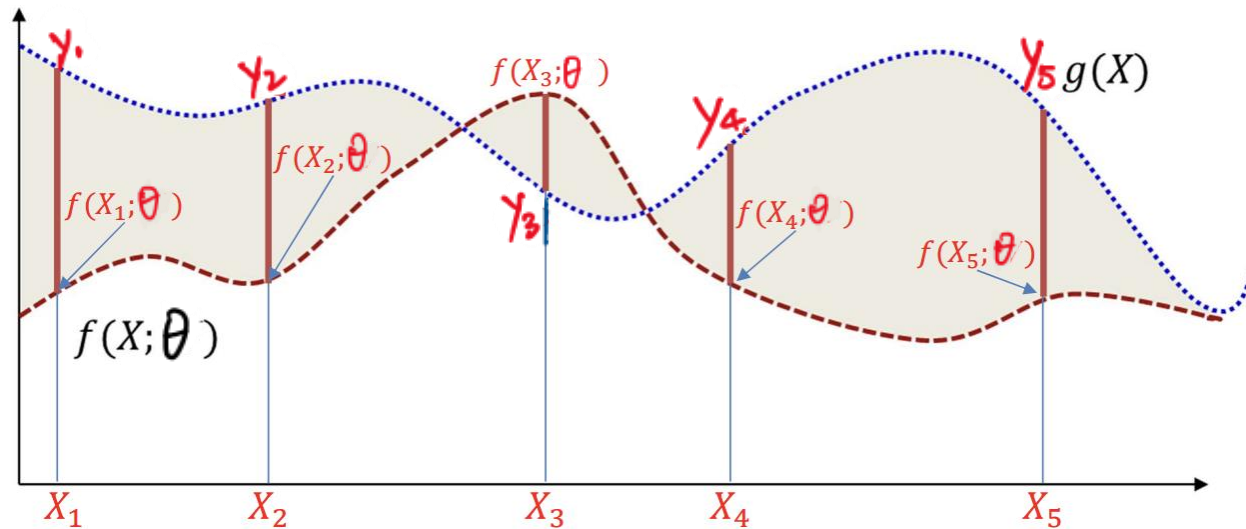
R^2 can be negative if your model is really bad!!

How to learn an ML model?



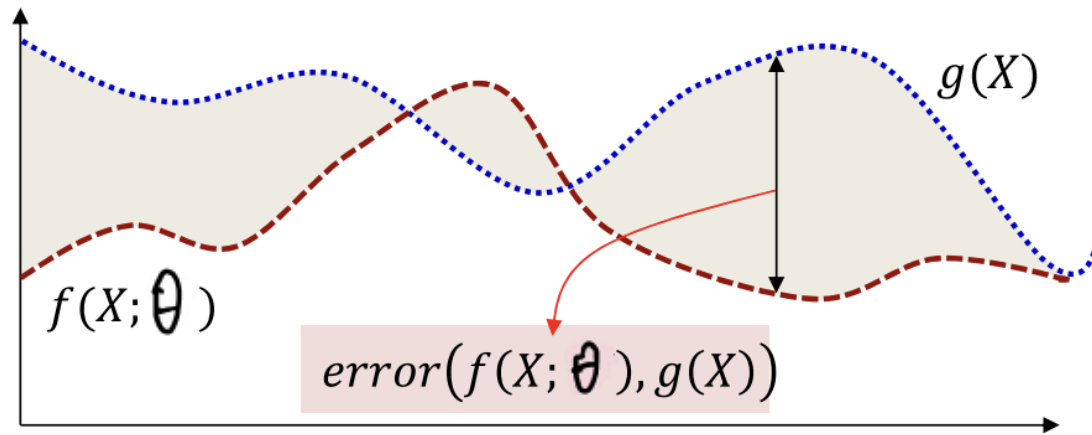
- The **blue curve** is the true data-generating mechanism that we are trying to learn
- **Red points**: data points contained in my dataset
 - Sampled from data generating function $g(X)$

How to learn an ML model?



- The **blue curve** is the true data-generating mechanism that we are trying to learn
- The **red curve** is the current function that I fitted on the data

How to learn a ML model?



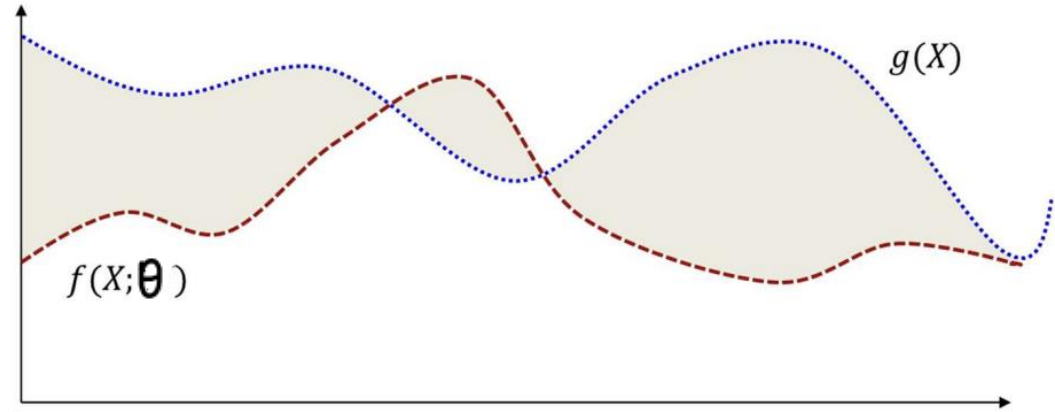
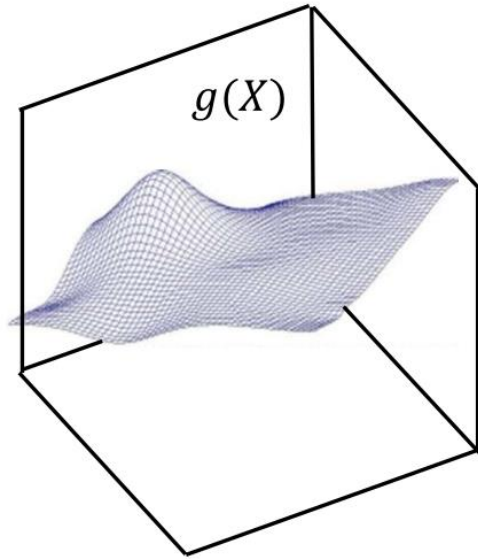
- The shaded area

$$totalerr(\theta) = \int_{-\infty}^{\infty} error(f(X; \theta), g(X)) dX$$

- The optimal θ

$$\widehat{\theta} = \underset{\theta}{\operatorname{argmin}} totalerr(\theta)$$

Problem: $g(X)$ is unknown

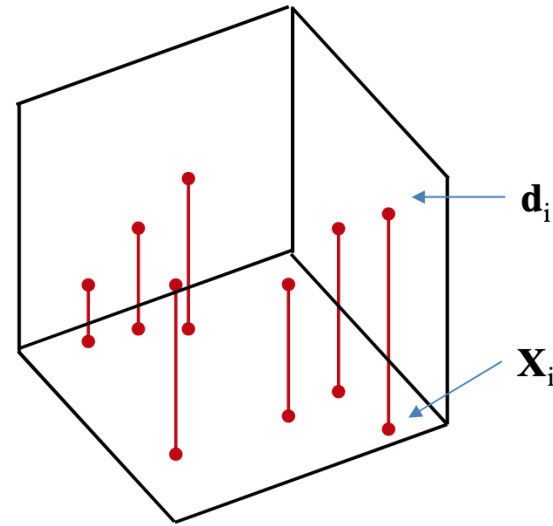
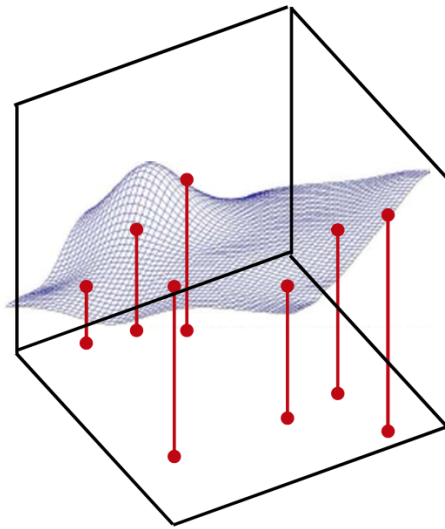


- Function must be fully specified in order to compute

$$\int_{-\infty}^{\infty} \text{error}(f(X; \theta), g(X)) dX$$

- In practice we will not have such specification

Solution: Sampling the function



- Sample $g(\mathbf{x})$
 - Basically, get input output pairs for a number of samples of input
 - Very easy to do in most problems: just gather training data
 - We must learn the entire function from these few examples
 - The “training” samples